

Experimental Empirical Analysis of Sorting Algorithms

Presentation Review Report

Presenter: Daniel Pantea

Reviewer: Maria Maiorescu

Faculty of Computer Science
West University of Timișoara

1 Main Results Presented

The presentation focused on an experimental comparison of six classical sorting algorithms: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort. The algorithms were implemented in C and tested on arrays of different sizes and input structures, including random, reversed, and partially sorted data.

The results showed that Bubble Sort and Selection Sort become inefficient for large inputs, while Merge Sort and Quick Sort remain efficient in most situations. Radix Sort achieved the best performance for large integer arrays. The presentation also highlighted how the structure of the input data can significantly influence practical performance, especially for Insertion Sort and Quick Sort.

2 What Worked Well in the Presentation

- The presentation was well organized and followed a logical structure.
- The objectives and methodology were clearly explained.
- The graphs helped visualize the performance differences between algorithms.
- The presenter successfully connected theoretical complexity with experimental results.
- The conclusions were supported by the collected data.

3 What Did Not Work Well in the Presentation

- Figure 2 from slide 9 is difficult to understand.
- The problems encountered are not detailed enough and the resolution is not clear.
- More examples of real-world applications could have made the topic even more engaging.

4 What I Learned from the Presentation

I learned that input structure has a major impact on the practical performance of sorting algorithms. In particular, Insertion Sort performs much better on nearly sorted data than its theoretical average-case complexity might suggest. I also learned that Radix Sort can outperform comparison-based algorithms when sorting large arrays of integers.

5 Questions Asked and Answers Received

Question: Did you have any issues when running the program?

Answer: Yes. The presenter initially encountered a segmentation fault while running Quick Sort because an incorrect pivot was chosen. The issue was solved by switching to the median-of-three pivot strategy, which improves the likelihood of obtaining balanced partitions and generally leads to better Quick Sort performance.